LAPORAN TUGAS KECIL 2

Algoritma Divide and Conquer untuk Voxelization Objek 3d menggunakan Octree

IF2211 STRATEGI ALGORITMA

SEMESTER II Tahun 2025/2026

Leonardus Brain Fatolosja 13524146

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Leonardus Brain Fatolosja

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

I Algoritma Divide and Conquer dalam melakukan *Voxelization*

Voxelization adalah proses konversi objek 3d ke dalam bentuk-bentuk voxel, yaitu kubus-kubus. Dalam program ini, voxelization dilakukan dengan menggunakan octree. Octree adalah struktur data tree yang berisi delapan (8) anak. Proses voxelization bisa memanfaatkan konsep octree dan diwujudkan dengan menggunakan divide and conquer. Dengan menggunakan divide and conquer, penggunaan struktur data octree pun dapat dimanfaatkan untuk proses *voxelization*.

Langkah-langkah dari Algoritma Divide and Conquer dalam program ini adalah sebagai berikut.

1. Base Condition

Algoritma divide and conquer akan berhenti ketika mencapai **base condition**, yaitu ketika bagian (octree) sudah memenuhi persyaratan (*depth*/kedalaman rekursif sudah mencapai target yang diinginkan **atau** octree tidak beririsan dengan objek 3d yang ingin dikonversi). Ketika sudah mencapai target yang diinginkan, jika octree tersebut beririsan dengan objek 3d, maka voxel tersebut (yang digambarkan sebagai node octree) akan dimasukkan sebagai hasil. Jika tidak beririsan, maka voxel tersebut akan diabaikan.

Voxel akan disimpan ke dalam suatu array berisi hasil-hasil voxel konversi. Proses memasukkan voxel ke dalam array dilakukan saat mencapai base condition untuk mempercepat algoritma.

2. Membagi octree ke dalam delapan anak. (*Divide*)

Octree merupakan representasi data dari voxel yang ada. Kita ingin membagi satu kubus menjadi delapan kubus kecil yang membentuk satu kubus tersebut. Proses ini akan menggunakan octree untuk representasi kubus/voxel. Dengan menggunakan konsep yang sama, octree akan dibagi ke dalam delapan anak tersebut dengan secara bersamaan mengubah nilai dari voxel, yaitu titik tengah, ukuran dari titik tengah ke sisi, dan keterangan beririsan atau tidaknya dengan objek 3d.

3. Mengecek apakah voxel beririsan dengan objek 3d.

Untuk mengubah objek 3d mesh menjadi bentuk voxel, kita harus membuat voxel-voxel yang beririsan dengan objek 3d tersebut. Idenya adalah ketika voxel sudah beririsan dengan objek 3d, maka voxel tersebut seharusnya merepresentasikan bagian objek 3d sebelumnya. Maka, agar proses voxelization berhasil, diperlukan adanya algoritma untuk mengecek beririsan atau tidaknya voxel dengan objek 3d.

Cara untuk menentukan keadaan beririsan dengan menggunakan AABB (Axis-Aligned Bounding Box). Untuk membandingkan permukaan segitiga (faces) dengan kubus (voxel), kita menganggap keduanya berada di dalam suatu kotak yang tidak berputar dan memiliki sumbu yang sejajar. Langkah-langkahnya adalah sebagai berikut.

- Melakukan tes sumbu pemisah, yaitu ketiga titik yang membangun segitiga (faces) akan dihitung proyeksinya terhadap arah axis. Kemudian, dari ketiga nilai tersebut, kita akan menyimpan nilai minimum dan maximum (interval). Menghitung nilai proyeksi voxel (kubus), yaitu dengan menghitung pusat voxel dengan ukuran setengah terhadap arah axis. Tujuannya untuk mendapatkan nilai r dan membangun sebuah interval [-r,r]. Melakukan tes terakhir. Terakhir, kita membandingkan nilai r dan nilai proyeksi tiga sudut segitiga terhadap arah axis. Hasilnya adalah ketika seluruh proyeksi segitiga (minimum) berada di kanan r atau seluruh proyeksi segitiga (maksimum) berada di kiri -r, maka dapat dipastikan bahwa segitiga tersebut tidak beririsan dengan voxel.
- Setelah membuat tes sumbu, maka kita akan membandingkan lebih lanjut dengan membandingkannya pada 3 sumbu, yaitu atas/bawah, delapan/belakang, kanan/kiri.
- Jika tes sumbu berhasil dilewatkan, maka proses membandingkan akan lanjut ke proses selanjutnya, yaitu tes untuk normal bidang segitiga. Kedua keadaan ini tidak boleh ada secara bersamaan, yaitu normal bidang segitiga bukan 0 dan tes sumbu titik-titik segitiga dengan sumbunya adalah normal bidang segitiga. Jika keduanya terpenuhi (ada), maka kedua objek tersebut terpisahkan.
- Terakhir, untuk kasus yang hanya terkena sedikit, caranya dengan melakukan pengecekan pada sumbu-sumbu hasil perkalian silang vektor rusuk segitiga dengan sumbu x, y, dan z. Sama seperti sebelumnya, jika hasil perkalian tersebut adalah nol, maka tes sumbu titik-titik segitiga dengan sumbunya adalah hasil perkalian tersebut haruslah menghasilkan hasil bahwa keduanya beririsan. Jika tidak, maka kedua objek tersebut terpisahkan.

Perlu jadi catatan bahwa, untuk membuktikan bahwa kedua objek itu terpisahkan hanya membutuhkan satu kondisi yang benar. Sehingga, jika program sudah mendeteksi adanya kondisi yang tidak terpenuhi, maka program dapat langsung menganggap kedua objek tersebut terpisahkan.

4. Melakukan proses yang sama pada anak octree. (*Conquer*)

Masing-masing anak dari octree yang beririsan dengan objek 3d akan melakukan algoritma yang sama. Proses ini akan dilakukan secara rekursif sampai mencapai base condition.

5. Hasil Akhir Algoritma

Pada base condition, akan dihasilkan sebuah array berisi voxel hasil konversi objek 3d. Kemudian, voxel ini akan diubah menjadi bentuk vertices(vertex/titik) dan faces (face/luas/sisi/2d) segitiga. Satu voxel akan menghasilkan 8 vertices dan 12 faces. Dengan menganggap sebuah voxel adalah kubus, maka kita bisa mendapatkan posisi 8 sudut dari titik pusat kubus dan 6 sisi yang terbentuk dari 8 sudut tersebut. Karena menggunakan face segitiga, maka 6 sisi persegi tersebut membutuhkan 12 sisi segitiga.

PseudoCode

Dari langkah-langkah dan penjelasan yang sudah dijabarkan, berikut adalah pseudocode untuk menggambar implementasi algoritma secara garis besar.

```
PROGRAM UTAMA
    baca nama file input (tanpa .obj)
    Root ← baca ../test/<nama>.obj  // Input File: isi vertices + faces
    hitung bounding box semua vertex
    pusat ← tengah bbox; ukuran_kubus ← max(range x,y,z) * margin
    buat OctreeNode akar (Voxel(pusat, ukuran_kubus))
    baca depth maksimum
    jalankan Root.algorithm(&akar, depth=0, target=depth)
    hasil ← Root.konversiVoxel()  // voxel → vertex + face kubus
    tulis ../test/<nama_out>.obj  // Output File
    cetak: jumlah voxel, vertex, face, statistik node per depth,
           node tidak ditelusuri, waktu, pesan file keluar


BACA FILE .obj
  untuk setiap baris:
    jika "v x y z" → simpan Point
    jika "f i j k" → buat Face dari 3 vertex (indeks 1-based)

TULIS FILE .obj
  untuk tiap vertex → baris "v x y z"
  untuk tiap face   → baris "f a b c"

OCTREE: bagi node
  untuk i = 0..7:
    hitung pusat anak (8 oktan)
    children[i] ← OctreeNode(Voxel(pusat_anak, ukuran/2))


Root.algorithm(node, depth, target)
  jika depth = target ATAU node kosong (tidak fill):
      jika node fill → simpan voxel node; return
  bagi node jadi 8 anak
  untuk tiap anak:
      anak.fill ← ada Face yang AABB(Face, Voxel_anak)?
  jika depth < batas_thread:
      Root.algorithm(anak, depth+1, target) untuk anak yang fill
  else:
      rekursi serial untuk anak yang fill
  (anak tidak fill → dihitung sebagai node tidak ditelusuri)

Root.konversiVoxel()
  untuk tiap voxel tersimpan:
      tambah 8 titik kubus + 12 segitiga (2 per sisi) ke Result
  return Result

isContain(voxel)
  untuk tiap Face di mesh:
      jika AABB(Face, voxel) → return true     // Penjelasan Ada di section sebelumnya
  return false
```

Code

Setelah membuat pseudocode, maka implementasi dalam bahasa pemrograman C++ pun bisa dilakukan. Code yang ditampilkan merupakan kode-kode yang berhubungan dengan penjelasan di atas (kode pembacaan file, penulisan file, dll tidak dimasukkan).

Pengecekan intersect - Algoritma Divide and Conquer

```
bool axisTest (Point Axis,Point P1, Point P2, Point P3, Point half){
    float f1 = Axis.dot(P1);
    float f2 = Axis.dot(P2);
    float f3 = Axis.dot(P3);
    float r = half.x*abs(Axis.x) + half.y*abs(Axis.y) + half.z*abs(Axis.z);
    float mi = min({f1,f2,f3});
    float ma = max({f1,f2,f3});
    return !(mi > r || ma < -r);
}

bool AABB(Face F, Voxel V){
    Point P1 = F.v1.sub(V.center);
    Point P2 = F.v2.sub(V.center);
    Point P3 = F.v3.sub(V.center);
    Point h = Point (V.size/2.0f,  V.size/2.0f, V.size/2.0f);

    // 1
    if (!axisTest({1,0,0}, P1,P2,P3, h)) return false;
    if (!axisTest({0,1,0}, P1,P2,P3, h)) return false;
    if (!axisTest({0,0,1}, P1,P2,P3, h)) return false;

    // 2
    Point e1 = P2.sub(P1), e2 = P3.sub(P2), e3 = P1.sub(P3);
    Point n = e1.cross(e2);
    if ((n.x!=0||n.y!=0||n.z!=0) && !axisTest(n, P1,P2,P3, h)) return false;

    // 3
    Point axes[3] = {{1,0,0},{0,1,0},{0,0,1}};
    for (Point ax : axes){
        Point a1 = e1.cross(ax);
        Point a2 = e2.cross(ax);
        Point a3 = e3.cross(ax);
        if ((a1.x!=0||a1.y!=0||a1.z!=0) && !axisTest(a1, P1,P2,P3, h)) return false;
        if ((a2.x!=0||a2.y!=0||a2.z!=0) && !axisTest(a2, P1,P2,P3, h)) return false;
        if ((a3.x!=0||a3.y!=0||a3.z!=0) && !axisTest(a3, P1,P2,P3, h)) return false;
    }

    return true;
}

```

```
void Root::algorithm(OctreeNode* octree, int depth, int target){
    // Base : Stop Rekursif kalau sudah kosong
    if (depth == target || octree->info.fill == false){
        if (octree->info.fill){
            lock_guard<mutex> lock(VoxelMutex);
            voxels.push_back(octree->info);
            VOXEL_COUNT++;
        }
        return;
    }else{
        // Rekursif : Bagi octree menjadi 8 bagian
        octree->divide();

        // Cek apakah ada faces yang masuk ke dalam voxel
        for (int i = 0; i < 8; i++){
            if (isContain(octree->children[i]->info)){
                octree->children[i]->info.fill = true;
            }else{
                octree->children[i]->info.fill = false;
            }
        }
        // Concurrency untuk tiap rekursif (Thread hanya sampai 3 sesuai dengan THREAD_LIMIT di dnc.hpp)
        if (depth < THREAD_LIMIT){
            thread threads[8];
            for (int i = 0; i < 8; i++){
                if (octree->children[i]->info.fill){
                    NODE_USED[depth]++;
                    threads[i] = thread(&Root::algorithm, this, octree->children[i], depth+1, target);
                }else{
                    NODE_UNUSED[depth]++;
                    threads[i] = thread([]{});
                }
            }
            for (int i = 0; i < 8; i++){
                threads[i].join();
            }
        }else{
            for (int i = 0; i < 8; i++){
                if (octree->children[i]->info.fill){
                    NODE_USED[depth]++;
                    algorithm(octree->children[i], depth+1, target);
                }else{NODE_UNUSED[depth]++;}
            }
        }
    }
}
```

```cpp
class Index{
public:
    int a,b,c;

    Index(int a, int b, int c) : a(a),b(b),c(c) {}
};

class Result{
public:
    // Hasil Vertices dan Faces
    vector<Point> vertices;
    vector<Index> faces;
};

class Root{
public:
    // Counter
    static int VOXEL_COUNT, VERTICES_COUNT, FACES_COUNT;
    static vector<int> NODE_USED, NODE_UNUSED;
    // DATA
    // Vektor berisi vertices
    vector<Point> vertices;
    // Vektor berisi faces
    vector<Face> faces;
    // Vektor berisi voxel
    vector<Voxel> voxels;

    // FUNCTIONS
    // Cek apakah faces ada di dalam voxel?
    bool isContain(Voxel v);

    // Pembagian octree secara rekursif
    void algorithm(OctreeNode* octree, int depth, int target);

    // Pengisian Voxel
    void fillVoxel(OctreeNode* octree, int depth, int target);

    // Konversi Voxel ke Faces dan Vertices
    Result konversiVoxel();
};
```

Pembagian Kubus menjadi 8 Kubus [Divide]

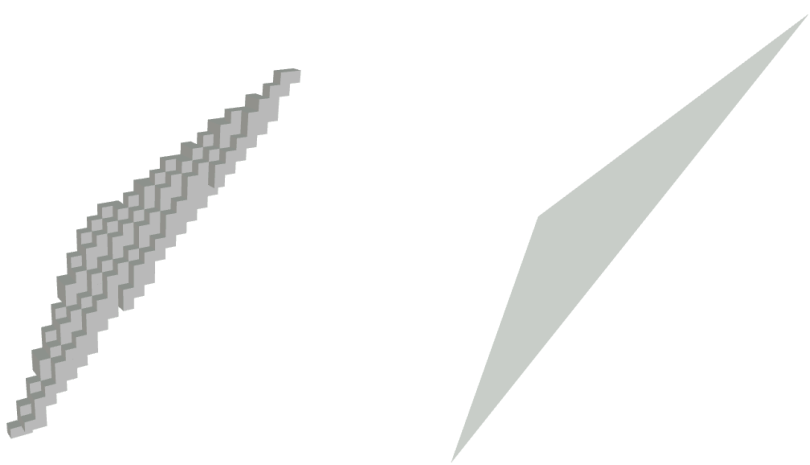
```cpp
void OctreeNode::divide(){
    float newSize = info.size/2.0f;
    float offset = info.size/4.0f;

    for(int i = 0; i < 8; i++){
        float ox = (i & 1) ? offset : -offset;
        float oy = (i & 2) ? offset : -offset;
        float oz = (i & 4) ? offset : -offset;

        Point childP = Point(info.center.x+ox, info.center.y+oy, info.center.z+oz);
        Voxel temp = Voxel(childP,info.size/2.0f, true);

        this->children[i] = new OctreeNode(temp);
    }
}
```

II Hasil Tes (Input dan Output)

1. Input 1: Line.obj (Depth 5)

```
 __     _____________.____   ________  _______     _____ __________.-.-.-
/  \   /  \_   _____/|    |  \_   ___ \ \_____ \   /     \ \_   _____/| | | |
\   \/\/   /|    __)_ |    |  /    \  \/  /   |  \  /  \ /  \ |    __)_ | | | |
 \        / |        \|    |__\    \____/    |   \/    Y    \|        \ \|\|\|
  \__/\  / /_______  /|_______ \_____  /\_______  /\____|__  /_______  / _____
       \/          \/         \/     \/         \/         \/       \/  \/\/\/

===PROGRAM VOXELIZATION===
[Ketentuan]
1. File .obj hanya berisi vertex dan face dengan format standar (v x y z untuk vertex dan f v1 v2 v3 untuk face)
2. Jika ada format yang dilanggar, program akan menghiraukannya
3. File .obj tidak boleh berisi string selain faces dan vertices: Hal ini akan mengakibatkan error di program ini. (Term
asuk tidak boleh ada kata pengantar di awal)

===[Default: File .obj disimpan pada folder test/]===
Pilihan Input Nama File:
1. File name only (tanpa .obj)
2. Path (Jika ingin memasukkan path lengkap berikut dengan .obj)
Masukkan opsi: 1
Masukkan nama file: line
Masukkan Depth: 5
Masukkan nama file akhir [Default: File name only]: line-result
```

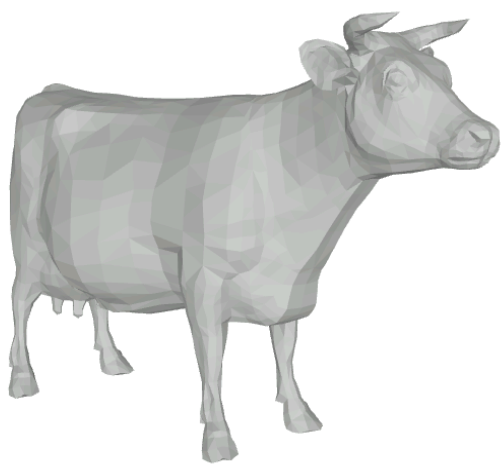
```
---INFOMRASI OUTPUT---
Banyaknya Voxel Yang terbentuk: 244
Banyaknya Vertex yang terbentuk: 1952
Banyaknya Faces yang terbentuk: 2928
Kedalaman Octree yang digunakan: 5
Banyaknya Node Octree yang terbentuk:
1: 8
2: 48
3: 112
4: 240
5: 632
Banyaknya Node Octree yang tidak ditelusuri:
1: 2
2: 34
3: 82
4: 161
5: 388
Waktu Komputasi Algoritma: 6.27374s

[File berhasil diproses dan disimpan sebagai line-result.obj di dalam folder test/]
```

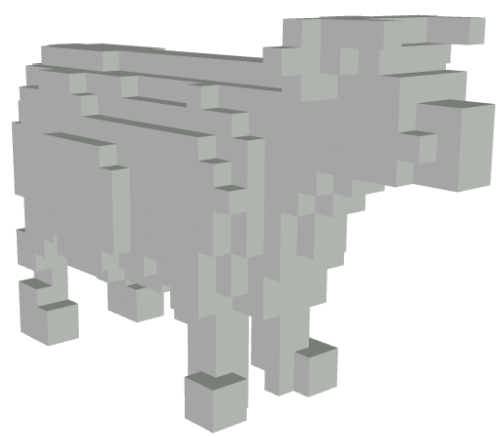


2. Input 2: Cow.obj (Depth 5)

```
 __      _____________.____   ________  _______      _____  __________._._._.
/  \    /  \_   _____/|    |  \_   ___ \ \_____  \    /     \ \_   _____/| | | |
\   \/\/   /|    __)_ |    |  /   \  \/  /   |   \  /  \ /  \ |    __)_ | | | |
\        / |        \|    |__\    \____/    |    \/    Y    \|        \ \|\|\|
\__/\  / /_______  /|_______ \______  /\_______  /\____|__  /_______  / _____
     \/          \/         \/      \/         \/         \/        \/  \/\/\/

===PROGRAM VOXELIZATION===
[Ketentuan]
1. File .obj hanya berisi vertex dan face dengan format standar (v x y z untuk vertex dan f v1 v2 v3 untuk face)
2. Jika ada format yang dilanggar, program akan menghiraukannya
3. File .obj tidak boleh berisi string selain faces dan vertices: Hal ini akan mengakibatkan error di program ini. (Term
asuk tidak boleh ada kata pengantar di awal)

===[Default: File .obj disimpan pada folder test/]===
Pilihan Input Nama File:
1. File name only (tanpa .obj)
2. Path (Jika ingin memasukkan path lengkap berikut dengan .obj)
Masukkan opsi: 1
Masukkan nama file: cow
Masukkan Depth: 5
Masukkan nama file akhir [Default: File name only]: cow-result
```

```
---INFOMRASI OUTPUT---
Banyaknya Voxel Yang terbentuk: 1128
Banyaknya Vertex yang terbentuk: 9024
Banyaknya Faces yang terbentuk: 13536
Kedalaman Octree yang digunakan: 5
Banyaknya Node Octree yang terbentuk:
1: 8
2: 64
3: 160
4: 592
5: 2272
Banyaknya Node Octree yang tidak ditelusuri:
1: 0
2: 44
3: 86
4: 308
5: 1144
Waktu Komputasi Algoritma: 2.68353s

[File berhasil diproses dan disimpan sebagai cow-result.obj di dalam folder test/]
```

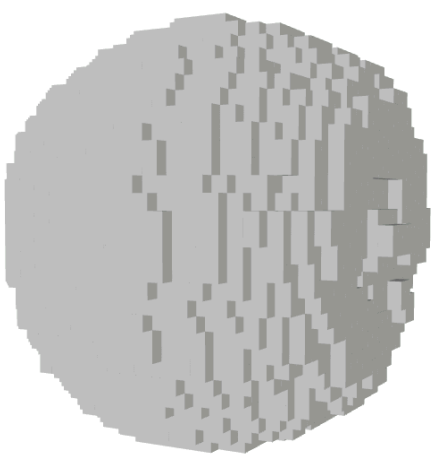


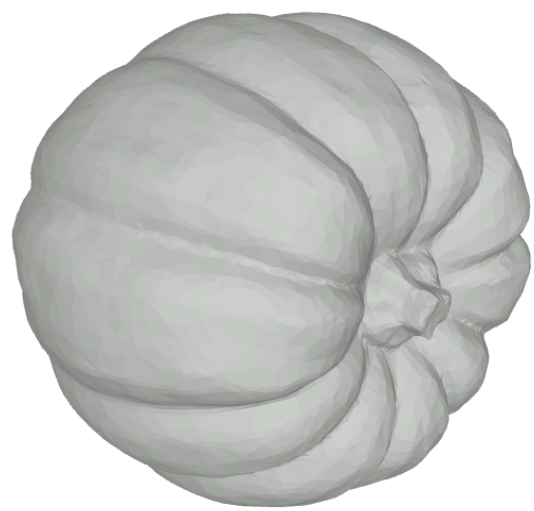


3. Input 3: Pumpkin.obj (Depth 5)

```
 __      _____________.____    ________  _______      _____  __________._.._.
/  \    /  \_   _____/|    |  \_   ___ \ \_____  \    /     \ \_   _____/| | | |
\   \/\/   /|    __)_ |    |  /    \  \/  /   |   \  /  \ /  \ |    __)_ | | | |
\        / |        \|    |__\    \____/    |    \/    Y    \|        \ \|\|\|
\__/\  / /_______  /|_______ \_____  /\_______  /\____|__  /_______  / _____
     \/          \/         \/     \/         \/         \/        \/  \/\/\/

===PROGRAM VOXELIZATION===
[Ketentuan]
1. File .obj hanya berisi vertex dan face dengan format standar (v x y z untuk vertex dan f v1 v2 v3 untuk face)
2. Jika ada format yang dilanggar, program akan menghiraukannya
3. File .obj tidak boleh berisi string selain faces dan vertices: Hal ini akan mengakibatkan error di program ini. (Term
asuk tidak boleh ada kata pengantar di awal)

===[Default: File .obj disimpan pada folder test/]===
Pilihan Input Nama File:
1. File name only (tanpa .obj)
2. Path (Jika ingin memasukkan path lengkap berikut dengan .obj)
Masukkan opsi: 1
Masukkan nama file: pumpkin
Masukkan Depth: 5
Masukkan nama file akhir [Default: File name only]: pumpkin
```

```
---INFOMRASI OUTPUT---
Banyaknya Voxel Yang terbentuk: 3849
Banyaknya Vertex yang terbentuk: 30792
Banyaknya Faces yang terbentuk: 46188
Kedalaman Octree yang digunakan: 5
Banyaknya Node Octree yang terbentuk:
1: 8
2: 64
3: 448
4: 1840
5: 7728
Banyaknya Node Octree yang tidak ditelusuri:
1: 0
2: 8
3: 218
4: 874
5: 3879
Waktu Komputasi Algoritma: 2.64049s

[File berhasil diproses dan disimpan sebagai pumpkin.obj di dalam folder test/]
```

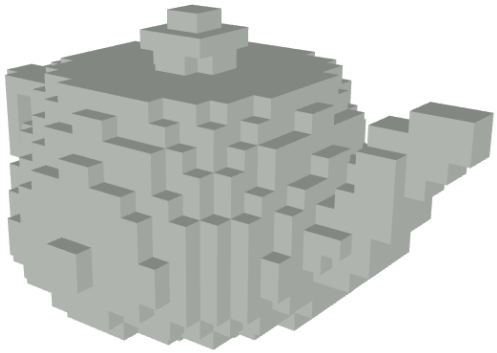




4. Input 4: teapot.obj (Depth 5)

```
 __      _____________.____    ________  _______      _____  __________.-.-.-.
/  \    /  \_   _____/|    |   \_   ___ \ \_____ \    /     \ \_   _____/| | | |
\   \/\/   /|    __)_ |    |   /    \  \/  /   |   \  /  \ /  \ |    __)_ | | | |
\        / |        \|    |__\    \____/   |    \/   Y    \|        \ \|\|\|
\__/\  / /_______  /|_______ \_____  /\_______  /\____|__  /_______  / _____
     \/          \/         \/     \/         \/         \/        \/  \/\/\/

===PROGRAM VOXELIZATION===
[Ketentuan]
1. File .obj hanya berisi vertex dan face dengan format standar (v x y z untuk vertex dan f v1 v2 v3 untuk face)
2. Jika ada format yang dilanggar, program akan menghiraukannya
3. File .obj tidak boleh berisi string selain faces dan vertices: Hal ini akan mengakibatkan error di program ini. (Term
asuk tidak boleh ada kata pengantar di awal)

===[Default: File .obj disimpan pada folder test/]===
Pilihan Input Nama File:
1. File name only (tanpa .obj)
2. Path (Jika ingin memasukkan path lengkap berikut dengan .obj)
Masukkan opsi: 2
Masukkan nama file: ../test/teapot.obj
Masukkan Depth: 5
Masukkan path file akhir [<path>/<filename>.obj]: ../test/teapot.obj
```

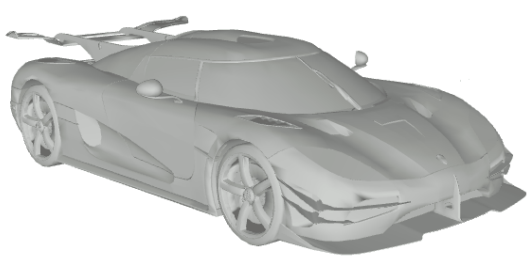
```
---INFOMRASI OUTPUT---
Banyaknya Voxel Yang terbentuk: 1530
Banyaknya Vertex yang terbentuk: 12240
Banyaknya Faces yang terbentuk: 18360
Kedalaman Octree yang digunakan: 5
Banyaknya Node Octree yang terbentuk:
1: 8
2: 64
3: 192
4: 768
5: 2992
Banyaknya Node Octree yang tidak ditelusuri:
1: 0
2: 40
3: 96
4: 394
5: 1462
Waktu Komputasi Algoritma: 1.33887s

[File berhasil diproses dan disimpan sebagai ../test/teapot.obj.obj di dalam folder test/]
```

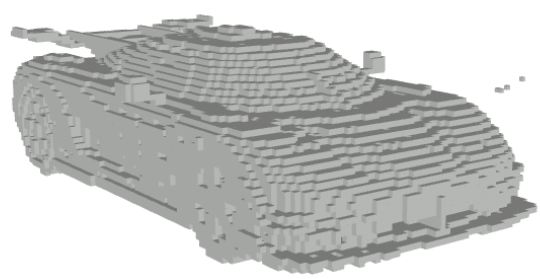


5. Input 5: Car.obj (Depth 7)

```
 __      _____________.____   ________  _______     _____  __________.-.-.-.
/  \    /  \_   _____/|    |  \_   ___ \ \_____  \    /     \ \_   _____/| | | |
\   \/\/   /|    __)_ |    |  /    \  \/  /   |   \  /  \ /  \ |    __)_ | | | |
\        / |        \|    |__\    \____/    |    \/    Y    \|        \ \|\|\|
\__/\  / /_______  /|_______ \______  /\_______  /\____|__  /_______  / ______
     \/          \/         \/      \/         \/         \/        \/  \/\/\/

===PROGRAM VOXELIZATION===
[Ketentuan]
1. File .obj hanya berisi vertex dan face dengan format standar (v x y z untuk vertex dan f v1 v2 v3 untuk face)
2. Jika ada format yang dilanggar, program akan menghiraukannya
3. File .obj tidak boleh berisi string selain faces dan vertices: Hal ini akan mengakibatkan error di program ini. (Term
asuk tidak boleh ada kata pengantar di awal)

===[Default: File .obj disimpan pada folder test/]===
Pilihan Input Nama File:
1. File name only (tanpa .obj)
2. Path (Jika ingin memasukkan path lengkap berikut dengan .obj)
Masukkan opsi: 1
Masukkan nama file: car
Masukkan Depth: 7
Masukkan nama file akhir [Default: File name only]:
```

```
---INFOMRASI OUTPUT---
Banyaknya Voxel Yang terbentuk: 32563
Banyaknya Vertex yang terbentuk: 260504
Banyaknya Faces yang terbentuk: 390756
Kedalaman Octree yang digunakan: 7
Banyaknya Node Octree yang terbentuk:
1: 8
2: 64
3: 208
4: 608
5: 2904
6: 14096
7: 63846
Banyaknya Node Octree yang tidak ditelusuri:
1: 0
2: 38
3: 132
4: 245
5: 1142
6: 6115
7: 31284
Waktu Komputasi Algoritma: 26.5334s

[File berhasil diproses dan disimpan sebagai car-result.obj di dalam folder test/]
```





III Lampiran

Pranala menuju *repository github*: https://github.com/leains/Tucil2_13524146

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Referensi:

- https://www.redblobgames.com/x/2112-obj-file-format/
- https://www.geeksforgeeks.org/dsa/octree-insertion-and-searching/
- https://medium.com/@sanjivjha/a-beginners-guide-to-3d-data-understanding-point-clouds-meshes-and-voxels-385e02108141
- https://medium.com/data-science/how-to-voxelize-meshes-and-point-clouds-in-python-ca94d403f81d
- https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection